the context to determine how many rows to create. One thing we need to deal with here is that the arrays being created by our Array.from trick start at zero, but the actual ticket data has both row and seats starting at 1.

For this code, we're converting both row and seat to the 1-based array as soon as we can:

chapter_11/01/app/javascript/components/venue_body.tsx

```
import * as React from "react"
import Row from "./row"
import { IsVenueContext, VenueContext } from "./app"

const rowItems = (rowCount) => {
  const rowNumbers = Array.from(Array(rowCount).keys())
  return rowNumbers.map((rowNumber) => (
    <Row key={rowNumber + 1} rowNumber={rowNumber + 1} />
  ))
}

export const VenueBody = (): React.ReactElement => {
  const context = React.useContext<IsVenueContext>(VenueContext)
  return (
    <table className="table">
      <tbody>{rowItems(context.state.rowCount)}</tbody>
    </table>
  )
}

export default VenueBody
```

The Row component is no longer tracking the state of its seats, so it can be much simpler:

chapter_11/01/app/javascript/components/row.tsx

```
import * as React from "react"
import Seat from "./seat"
import { IsVenueContext, VenueContext } from "./app"

interface RowProps {
  rowNumber: number
```